

Module 9 — LEGO Collection Catalog

1. Purpose and Scope

Catalog the household's LEGO sets so that, at any moment, the user can scroll through everything they own and know — per physical copy — **where it is, what condition it's in, what it cost, and what it's worth today.**

The module distinguishes the **catalog identity** of a set (**LegoSetModel** — its official metadata and current market value, shared by every copy owned) from each **physical copy** owned (**LegoSetInstance** — its own cost, storage location, build state, condition and ownership status). This is the one piece of structure the module keeps, because it is what makes "I own three copies of 10307, and this one is in Garagem › Caixa TV, sealed" expressible.

Everything else is deliberately kept flat. This is a household tool for a 4-person home, not an asset-management platform: **three tables, no background jobs, no scraper, no valuation engine.** Where a table could be replaced by a field, it was.

All monetary fields are EUR only. Performance metrics are **unrealized** — current market value versus acquisition cost. Sale price and date are recorded for history but do not drive KPIs.

Scope is strictly LEGO. Future support for other collectible types must not influence the initial domain model or user interface.

Actors

Primary Actor

A member of the household who owns LEGO sets and wants to browse the collection, find where any copy is stored, record what it cost and what it's worth, and keep a light history of copies that left the collection.

User Stories

US01. Set Registration via Lookup

As a user, I want to add a set by typing its set number so the system pre-fills name, theme, year, piece count, minifig count, retirement year and cover image from Brickset — and, if the lookup fails or the set is a custom/MOC build, I want to type everything myself using the same form.

US02. Per-Copy Tracking

As a user, I want each physical copy to carry its own acquisition cost and date, storage location, build state, condition, box/instructions presence and notes, so I know the exact status and whereabouts of every item I own.

US03. Browse, Search & Filter

As a user, I want to scroll my whole collection and narrow it down by set number, name, theme, storage location, build state, condition or ownership status.

US04. Current Value & Unrealized ROI

As a user, I want to record the current market value of a set by hand, refresh it whenever I feel like it, and see unrealized gain and ROI — with a clear indication of how stale the value is.

US05. Collection Analytics

As a user, I want a small overview of total cost, current value, ROI, number of sets, copies and pieces, and a breakdown by theme.

US06. Storage Awareness

As a user, I want to know what is inside each box and how full each box is, using my own eyeball estimate of fullness.

US07. Images

As a user, I want one picture per set and, optionally, one photo of my own copy, stored locally on my NAS so the collection stays browsable even if the source website changes.

US08. Simple Lifecycle

As a user, I want to mark a copy as **SOLD** or **GIFTED** to keep it in history, delete it outright when it was a mistake, and have departed copies excluded from collection value.

US09. Purchase Link

As a user, I want to optionally link a copy to the bank transaction that paid for it, so I can jump straight to the statement line and see when I bought it.

Data Model

Entities & Attributes

All monetary attributes are EUR only (no multi-currency, no FX). Stored as **NUMERIC(10,2)** decimal EUR (e.g. **19.99**); UI displays pt-PT-formatted (e.g. **19,99€**).

Every row carries **entity_id** — the owning household entity (an individual member, or a joint entity such as the couple). Entity is an **attribution and filter** dimension, not a security boundary: every household member can read every entity's LEGO data (see M7).

Definition — in-collection instance. Unless stated otherwise, every KPI and aggregate in this module operates over **LegoSetInstance** rows where **is_deleted = false AND ownership_status = 'IN_COLLECTION'**. **SOLD/GIFTED** copies stay visible in history but are excluded from cost, value, ROI and counts.

1. **LegoSetModel** (Catalog Identity + Current Value)

One row per unique **set_number** (or per custom build) within an entity's scope. Every copy owned points at it.

Attribute	Type	Description
<code>id</code>	UUID	Primary key
<code>entity_id</code>	UUID	Owning entity (attribution/filter)
<code>set_number</code>	String?	Official LEGO set number (e.g. "10307"); NULL for custom/MOC builds
<code>is_custom</code>	Boolean	true for a custom/MOC build with no official set number (default: false)
<code>name</code>	String	Set name
<code>theme</code>	String?	Primary theme (e.g. "Icons", "Star Wars")
<code>subtheme</code>	String?	Optional subtheme
<code>release_year</code>	Integer?	Year released
<code>retired_year</code>	Integer?	Year retired (NULL if still available)
<code>piece_count</code>	Integer?	Total pieces
<code>minifig_count</code>	Integer?	Total minifigures
<code>rrp_eur</code>	NUMERIC(10,2)?	Original recommended retail price, EUR (from Brickset; informational)
<code>current_value_eur</code>	NUMERIC(10,2)?	Current market value, EUR — manually maintained
<code>value_updated_at</code>	Date?	When <code>current_value_eur</code> was last set by the user
<code>image_document_id</code>	UUID?	Cover image, stored locally (Core Document)
<code>short_description</code>	String?	Concise 1-sentence summary
<code>notes</code>	Text?	Free-text notes about the set itself
<code>is_deleted</code>	Boolean	Soft-delete flag (default: false)
<code>created_at / updated_at / deleted_at?</code>	Timestamp	Audit timestamps

Unique constraint: (`entity_id`, `set_number`) when `set_number` IS NOT NULL.

No valuation history table. `current_value_eur` is a single hand-maintained figure per set, updated whenever the user feels like it. Its change history is recoverable from **AuditLog** if ever needed. The accepted trade-off: a sealed copy and a battered used copy of the same set are valued identically — acceptable at household scale, and avoidable by simply entering the value you'd actually get.

2. LegoSetInstance (Owned Physical Copy)

Exactly **one** physical copy per row. There is no **quantity** field — owning three copies means three rows pointing at the same **LegoSetModel**.

Attribute	Type	Description
id	UUID	Primary key
entity_id	UUID	Owning entity
lego_set_model_id	UUID	Foreign key to LegoSetModel
acquisition_date	Date?	Date acquired
acquisition_cost_eur	NUMERIC (10,2)	Price actually paid, EUR; 0.00 for gifts
acquisition_source	Enum?	RETAIL SECONDHAND GIFT OTHER
acquisition_transaction_id	UUID?	Optional direct FK to the bank Transaction (M2) that paid for it
storage_location_id	UUID?	Foreign key to StorageLocation
build_state	Enum?	SEALED BUILT DISASSEMBLED
condition	Enum?	NEW GOOD WORN DAMAGED — overall physical state of box and pieces
has_box	Boolean	Original box present (default: true)
has_instructions	Boolean	Instruction booklet present (default: true)
missing_parts	Text?	Free-text list of missing pieces (e.g. " 2x 3001 vermelho, 1x canopy "). Empty/ NULL = complete
ownership_status	Enum	IN_COLLECTION SOLD GIFTED (default: IN_COLLECTION)
sale_price_eur	NUMERIC (10,2)?	Price received; only meaningful when SOLD
sale_date	Date?	Date sold or given away
photo_document_id	UUID?	One photo of this exact copy, stored locally (Core Document)
notes	Text?	Free-text notes about this copy
is_deleted	Boolean	Soft-delete flag (default: false)
created_at / updated_at / deleted_at?	Timestamp	Audit timestamps

Constraints & Rules:

- acquisition_cost_eur** is required and non-negative (**0.00** for gifts).

- `sale_price_eur` / `sale_date` are optional even when **SOLD** — the user may not remember or care. No DB-level check ties them to `ownership_status`.
- Moving a copy back to **IN_COLLECTION** clears `sale_price_eur` and `sale_date`.
- `missing_parts` replaces a structured missing-parts table entirely. It does **not** affect `current_value_eur`.

3. StorageLocation (Flat Two-Level Place)

A flat list — **no parent/child tree**. Real usage is always `area` + `container` (`Garagem` / `Caixa TV`, `Casa` / `Armário`, `Casa` / `A uso`, `Casa` / `Montado`, `Garagem` / `Caixa A`), displayed as `Garagem > Caixa TV`.

Attribute	Type	Description
<code>id</code>	UUID	Primary key
<code>entity_id</code>	UUID	Owning entity
<code>area</code>	String	Room or zone (e.g. " <code>Garagem</code> ", " <code>Casa</code> ")
<code>container</code>	String?	Box, shelf or state within the area (e.g. " <code>Caixa TV</code> ", " <code>Armário</code> ", " <code>A uso</code> ")
<code>description</code>	String?	Optional access/condition details
<code>capacity_pct</code>	Integer?	Hand-annotated fullness estimate, 0–100 . NULL = not tracked
<code>is_deleted</code>	Boolean	Soft-delete flag (default: false)
<code>deleted_at</code>	Timestamp?	Soft-delete timestamp

Unique constraint: (`entity_id`, `area`, `container`).

Images

No `LegoImage` table. Images reuse the Core `Document` entity via two nullable FKs (`LegoSetModel.image_document_id`, `LegoSetInstance.photo_document_id`):

- **One image per set model, one photo per instance.** No galleries, no primary flag, no image-kind enum.
- **Always stored locally.** When the source is a URL (the 90% case — a Brickset cover image), the server downloads it **once**, validates magic bytes, and persists the bytes on the NAS volume. `Document.url` keeps the original address purely as provenance; it is never fetched again at render time. Direct uploads join the same code path from the validate-and-write step onward.
- Served through the standard authenticated, signed, time-limited document route already used by receipts — no LEGO-specific serving path.
- The instance's own photo takes display precedence over the model's cover image.

External Links

No **ExternalListing** table. BrickLink, BrickEconomy, Brickset and Rebrickable URLs are deterministic from **set_number** and are built client-side from a template constant. Nothing is stored, nothing is fetched, nothing goes stale.

Computed & Derived Fields

Evaluated on read; nothing is persisted.

- **owned_copies_count** (per model) = **COUNT(instances)** where in-collection
- **current_value_eur** (per instance) = its model's **current_value_eur**
- **appreciation_eur** (per instance) = **current_value_eur - acquisition_cost_eur**
- **roi_pct** = (**appreciation_eur / acquisition_cost_eur**) * 100; **NULL** when **acquisition_cost_eur = 0** (gifts) or the model has no **current_value_eur**
- **value_is_stale** (per model) = **value_updated_at** older than a configurable threshold (default 180 days) or **NULL**
- **is_retired** (per model) = **retired_year IS NOT NULL**
- **is_complete** (per instance) = **missing_parts IS NULL OR missing_parts = ''**
- **stored_count** (per location) = **COUNT(instances)** in-collection at that location — informational, independent of **capacity_pct**
- **remaining_capacity_pct** = **100 - capacity_pct** (**NULL** if untracked); **is_full** = **capacity_pct >= 100**

Functional Requirements

- **FR-9.1 Set Model CRUD.** Create, edit, soft-delete and hard-delete **LegoSetModel** rows. Find-or-create by (**entity_id**, **set_number**) when adding a copy, so repeat copies reuse the same model row.
- **FR-9.2 Brickset Lookup.** On entering a **set_number**, pre-fill name, theme, subtheme, release year, retired year, piece count, minifig count, **rrp_eur** and cover image from **Brickset** — the single configured provider (free API key in **Setting**). Every pre-filled field stays editable and manual edits are never overwritten by a later refresh. Lookup failure is not a dead end: the user lands in the same form with empty fields and completes it by hand. Custom/MOC builds (**is_custom = true**) skip lookup entirely. Brickset does **not** supply reliable market value — **current_value_eur** is always manual.
- **FR-9.3 Instance CRUD.** Create, edit, soft-delete and hard-delete **LegoSetInstance** rows, one per physical copy. Minimal required input: a model reference (set number, or a name for a custom build) plus **acquisition_cost_eur**. Everything else is optional or defaulted.
- **FR-9.4 Copy Grouping.** For any model, expose **owned_copies_count** and the list of its copies, so the collection can be browsed either flat (one row per copy) or grouped by set.
- **FR-9.5 Purchase Link.** Optionally set **acquisition_transaction_id** to the bank **Transaction** (M2) that paid for the copy, so the user can jump straight to the statement line. Selection uses the **shared transaction picker** (see UX-9.7), not a raw ledger scroll. Absent link is perfectly valid.
- **FR-9.6 Manual Current Value.** Let the user set **current_value_eur** on a model at any time; stamp **value_updated_at** automatically. Surface staleness in the UI rather than hiding the value. No provider refresh, no scheduled job, no snapshot table.
- **FR-9.7 Storage Locations.** Flat CRUD over **area + container**. Answer both "where is this set?" and "what is in this box?". **capacity_pct** is set by hand (quick presets 0/25/50/75/100 plus free entry)

and never computed; `remaining_capacity_pct` and `is_full` derive from it; locations are filterable/sortable by fullness.

- **FR-9.8 Condition & Completeness.** Track `build_state`, `condition`, `has_box`, `has_instructions` and free-text `missing_parts` per copy. `build_state` is an independently filterable field even when a storage location happens to describe a similar idea (e.g. `Casa` > `Montado`) — "show me everything sealed, wherever it is" must work.
- **FR-9.9 Ownership Lifecycle.** Three statuses only: `IN_COLLECTION`, `SOLD`, `GIFTED`. Any status may move to any other. `SOLD/GIFTED` copies are excluded from all value/ROI/count KPIs but remain browsable in history. Hard delete is offered alongside soft delete for genuine mistakes. Every create, delete and status change is written to `AuditLog`.
- **FR-9.10 External Links.** Render BrickLink / BrickEconomy / Brickset links built from `set_number` via a URL template; one click opens them. Never stored, never scraped, never refreshed.
- **FR-9.11 Images.** One locally stored cover image per model and one locally stored photo per instance, ingested either from a URL (downloaded once) or a direct upload, magic-byte validated, served via the shared signed document route.
- **FR-9.12 Collection Analytics.** Total cost, total current value, unrealized gain and ROI, unique sets, copies owned, total pieces, retired-set count, and a value/count breakdown by theme — all over in-collection copies.
- **FR-9.13 Discovery.** Search, filter, sort and paginate across set number, name, theme, storage location, build state, condition, completeness, ownership status, retirement status, acquisition cost, current value, piece count and minifig count.

UI / Screens

Design intent: modern, dense-but-readable, and progressive. Secondary detail lives in modals, side sheets, popovers and collapsible sections — never in a wall of fields. Every destructive or lifecycle action is one click away with a confirm.

- **UX-9.1 Two Areas.** `Overview` (metrics) and `Coleção` (the catalog). Storage is a side sheet reachable from both, not a third top-level tab.
- **UX-9.2 Overview.** Default landing view.
 - **KPI cards:** Total Cost · Current Value · Unrealized ROI % · Unique Sets · Copies Owned · Total Pieces · Retired Sets. Each card links into `Coleção` with the matching filter pre-applied.
 - **Charts:** value and copy count by theme (the one chart that earns its keep). A "valores atualizados a ..." line states the oldest `value_updated_at` in the collection, with a shortcut to the sets whose value is stale.
 - No historical time series — there is no snapshot table to draw one from, by design.
- **UX-9.3 Collection Workspace.** A data-first grid, one row per copy, with an optional "agrupar por conjunto" toggle that rolls copies together and shows the copy count.
 - **Default columns (kept deliberately few):** thumbnail · set number · name · theme · storage location · build state · condition · acquisition cost · current value · ROI.
 - **Behind progressive disclosure:** subtheme, release/retired year, minifig count, RRP, notes, missing parts, images, transaction link, ownership history. Row expansion or a detail side sheet — not extra columns.
 - **Discovery:** one global search box (set number, name, description, notes) plus a combinable filter bar; sorting and pagination; density toggle.

- **Row actions:** view, edit, change storage, change ownership status, delete — via a compact action menu.
- **UX-9.4 Add / Edit.** An **Adicionar conjunto** action is available from both areas.
 - Starts with **set_number**; a "não tem número / é um MOC" escape hatch switches to free-form entry.
 - Lookup runs on demand and fills what it can; results are always editable; a failed lookup silently degrades into the manual form with an inline notice.
 - If the set number already exists, the user is offered "adicionar outra cópia" instead of creating a duplicate model.
 - Acquisition cost, date, source, storage location, build state, condition, box/instructions, missing parts, notes and photo are all completable before saving, in a single modal with collapsed advanced sections.
- **UX-9.5 Set / Copy Detail.** A side sheet, not a page navigation:
 - Identity and metadata, cover image, external links.
 - Current value with **value_updated_at** and an inline "atualizar valor" control.
 - Per-copy cost, appreciation, ROI, storage, build state, condition, box/instructions, missing-parts text, notes, own photo.
 - Sale price/date when **SOLD**, shown apart from ROI.
 - Link to the bank transaction, when set.
- **UX-9.6 Storage.** A flat list grouped by **area**, each container showing **stored_count**, a **capacity_pct** slider with 0/25/50/75/100 presets, **remaining_capacity_pct** and a full/near-full badge. Selecting a container lists its contents. Filter/sort by fullness.
- **UX-9.7 Shared Transaction Picker.** Linking a purchase opens a reusable picker that proposes bank transactions near **acquisition_date** with a similar amount, ranked, with free search as a fallback. This component is **shared** with M1/M3/M4/M5 — it is specified here only because M9 is built first; it belongs to the shared frontend layer.

Proposed API Surface

Indicative; may be refined during implementation as long as the capabilities and rules above are preserved. Routes are namespaced **/lego** — the module is LEGO-only by design.

1. Overview

- **GET /lego/overview** — KPI totals, theme breakdown, stale-value summary.

2. Set Models

- **GET /lego/models** — list/search/filter, with **owned_copies_count**.
- **POST /lego/models** — create manually or as a custom/MOC build.
- **POST /lego/models/lookup** — Brickset metadata lookup by set number; read-only, mutates nothing.
- **GET /lego/models/{modelId}** — details plus its copies.
- **PATCH /lego/models/{modelId}** — update metadata or **current_value_eur** (stamps **value_updated_at**).
- **DELETE /lego/models/{modelId}** — soft-delete (**?hard=true** for hard delete). Blocked while any non-deleted copy references it.

- **PUT /lego/models/{modelId}/image** — set the cover image from a URL (downloaded and stored locally) or an upload.

3. Copies

- **GET /lego/instances** — flat, paginated, multi-facet filtering.
- **POST /lego/instances** — register a copy; find-or-creates the parent model from **set_number** when needed.
- **GET /lego/instances/{instanceId}** — full detail.
- **PATCH /lego/instances/{instanceId}** — update any field, including **ownership_status**.
- **DELETE /lego/instances/{instanceId}** — soft-delete, or hard delete with **?hard=true**.
- **PUT /lego/instances/{instanceId}/photo** — set this copy's photo from a URL or an upload.

4. Storage

- **GET /lego/storage-locations** — flat list with **stored_count**, **capacity_pct**, **remaining_capacity_pct**, **is_full**.
- **POST /lego/storage-locations** · **PATCH /lego/storage-locations/{locationId}** · **DELETE /lego/storage-locations/{locationId}** (blocked while in-collection copies are assigned).
- **GET /lego/storage-locations/{locationId}** — location plus its contents.

Analytics & KPIs

Over in-collection copies only:

- Σ acquisition cost, Σ current value, unrealized gain, overall unrealized ROI %.
- Per-copy appreciation and ROI; top gainers / losers.
- Value and copy count by theme.
- Unique sets, copies owned, total pieces, retired-set count.
- Storage: copies and value per location; count of full / near-full locations.
- Reported separately, never mixed into ROI: count of **SOLD/GIFTED** copies and Σ **sale_price_eur**.

Edge Cases & Business Rules

- **Single currency.** Every monetary attribute is EUR. No **currency** field, no FX logic anywhere in this module.
- **Gifts.** **acquisition_cost_eur = 0.00**; **roi_pct** is **NULL** (no division by zero); the UI shows — rather than a fake percentage.
- **No current value yet.** A model without **current_value_eur** contributes its acquisition cost to totals but is excluded from the ROI numerator/denominator, and is listed under "sem valor definido".
- **Stale value.** Values older than the configured threshold are flagged in the UI; the number is still used, never silently zeroed.
- **Custom builds / MOCs.** **set_number = NULL**, **is_custom = true**, lookup disabled, external links hidden.
- **Multiple copies.** Each copy is its own row with independent cost, condition and location; **owned_copies_count** is always derived.
- **Lifecycle.** Any transition between **IN_COLLECTION**, **SOLD** and **GIFTED** is allowed and audited. Returning to **IN_COLLECTION** clears sale fields. **SOLD/GIFTED** copies drop out of all KPIs but remain

browsable.

- **Deletion.** Soft delete keeps history; hard delete is available for genuine mistakes and is audited. Deleting a model is blocked while any non-deleted copy references it.
- **Storage deletion guard.** Deleting a location is blocked while in-collection copies are assigned to it; they must be moved or unassigned first.
- **Over-capacity.** Assigning a copy to a location with `is_full = true` warns in the UI but never blocks the save. `capacity_pct` never changes automatically.
- **Missing parts.** Free text only. It flags the copy as incomplete in listings and filters, and deliberately does **not** adjust `current_value_eur`.
- **Images.** Remote images are downloaded once, magic-byte validated and stored locally; a dead source URL never breaks the collection view. Uploads follow the identical validation path.

Deferred (explicitly not built now)

- Structured missing-part inventory with per-part costs and BrickLink part links.
- Valuation history, valuation bands (sealed / used / parts-out) and any automatic price refresh.
- Storage hierarchy beyond `area` + `container`.
- Image galleries, multiple photos per copy.
- Barcode/QR box scanning for fast entry.
- Contribution to the M6 net-worth roll-up — **intentionally out of scope**; the collection reports its own value inside this module only.

Definition of Done

- ☐ Three tables only: `LegoSetModel`, `LegoSetInstance`, `StorageLocation`.
- ☐ Model CRUD with find-or-create by `(entity_id, set_number)`; delete blocked while non-deleted copies reference it; hard delete available.
- ☐ Instance CRUD (minimal input: model reference + `acquisition_cost_eur`); no `quantity` field; soft and hard delete both work and are audited.
- ☐ Brickset lookup pre-fills metadata, never overwrites manual edits, and degrades cleanly to a manual form on failure or for custom builds.
- ☐ `current_value_eur` is manually settable, stamps `value_updated_at`, and staleness is surfaced in the UI.
- ☐ Unrealized appreciation/ROI computed correctly, including `NULL` for gifts and for models without a current value; sale figures excluded from ROI math.
- ☐ Ownership lifecycle (`IN_COLLECTION` / `SOLD` / `GIFTED`) enforced; non-in-collection copies excluded from every KPI; transitions audited; returning to `IN_COLLECTION` clears sale fields.
- ☐ Flat storage locations (`area` + `container`) CRUD; `capacity_pct` user-editable and never auto-computed; `remaining_capacity_pct` / `is_full` derived; "contents of container" and "where is this set" both work.
- ☐ `build_state`, `condition`, `has_box`, `has_instructions` and free-text `missing_parts` tracked and filterable.
- ☐ External links rendered from a `set_number` template; nothing stored or fetched.
- ☐ Images: one per model, one per instance; URL sources downloaded once and stored locally; magic-byte validated; served via the shared signed document route.
- ☐ Optional purchase link to a bank `Transaction` (M2) via the shared transaction picker.
- ☐ Analytics render (totals, ROI, theme breakdown); sale figures shown separately.

- ☐ Seed: a few example sets (e.g. 10307 Eiffel Tower, 75192 Millennium Falcon) with cost, current value, storage location and one image each.
- ☐ Tests, limited to what is genuinely load-bearing: ROI/appreciation math including the gift and no-value cases, ownership-transition guards, and storage capacity math.

Open Questions / Decisions (recorded during implementation)

#	Question	Decision	ADR
1	How to honour the optional purchase link (FR-9.5) before M2 exists?	Ship the contract, defer the constraint: <code>acquisition_transaction_id</code> is a nullable UUID column with no FK; <code>GET /transactions/suggest</code> answers <code>ledger_available: false</code> ; the shared picker is built and explains itself. The FK lands with M2.	0005
2	Where does <code>current_value_eur</code> history live, given there is no valuation table?	In <code>AuditLog</code> , which already snapshots <code>before/after</code> on every mutation. Staleness is surfaced (<code>value_is_stale</code> , <code>value_age_days</code> , amber badge) rather than hidden; no time-series chart is drawn.	0008
3	How is <code>(entity_id, set_number)</code> uniqueness enforced given soft delete and MOCs?	A partial unique index on <code>(entity_id, set_number)</code> <code>WHERE set_number IS NOT NULL AND is_deleted = false</code> , so soft-deleted rows and custom builds never collide.	—
4	Which entity owns a new copy when the selector is on «todas»?	Refused, never guessed — the add dialog asks for the entity inline.	0007
5	Brickset default state?	Off. The toggle and API key live in <code>Setting</code> ; the provider degrades to an explanatory message and the identical manual form. The key is never echoed back to the browser.	—
6	Which model does a duplicate <code>set_number</code> resolve to?	<code>POST /lego/models</code> answers <code>409 with model_id</code> , so the UI can offer «adicionar outra cópia» instead of creating a duplicate. <code>POST /lego/instances</code> find-or-creates silently.	—

Integration Contract

- **Exposes:** nothing to other modules. The collection's value is reported inside M9 only; it deliberately does **not** roll into M6 net worth. Dashboards (M8) may deep-link to the module.
- **Consumes:** `Entity` (attribution), `Transaction` (M2, optional purchase link), `Document` (locally stored images), `Setting` (Brickset API key, stale-value threshold), `AuditLog` (mutations).
- **Guarantees:** Brickset is contacted only on explicit user action; all monetary values are EUR-only; remote images are copied locally, never hotlinked.